Exploring Clustering Techniques for Flower Images

Ankan Roy

1 Introduction

In many real-world applications, large collections of images are easy to obtain but difficult and expensive to label. This project is inspired by the question: can we let an algorithm discover meaningful structure in unlabeled flower images? By focusing on unsupervised learning techniques, I aim to explore how clustering techniques can group flowers together based on their visual similarity and features.

An inspiration is the practical uses of unsupervised clustering. If visually similar flowers can be grouped reliably, this approach could be used as a preprocessing step for building better labeled datasets, reducing manual effort by letting annotators label clusters instead of individual images. The project is mainly inspired by the desire to understand how different image features drive clustering behavior. By using Python imaging or computer vision tools, we can experiment with resizing, color space transformations, and handcrafted feature extraction to represent each image as a feature vector. Feeding these vectors into K-Means lets me study how choices in preprocessing and feature design affect cluster quality.

Finally, the project showcases qualitative evaluation in unsupervised learning. The silhouette coefficient metric is used to judge how well the clusters are separated, complemented with visual inspection of clustered images, helping bridge the gap between numerical scores and human intuition.

2 Dataset

2.1 Data Source

For this experiment, the tf_flowers dataset from Tensorflow is used. This dataset is ideal for this experiment, as it contains thousands of images of common flowers (daisy, dandelion, rose, sunflower, tulip), each with many color and shape variations. Even though the dataset comes with labels, we can ignore them at training time to treat the data as unlabeled, where our goal is to organize them into clusters.

2.2 Data Preparation

The workflow begins by loading the dataset and converting it into a Pandas DataFrame, making it convenient to iterate through images with familiar tabular operations. Numerical class labels are mapped to human-readable flower names (e.g., 0 maps to “Dandelion,” 1 maps to “Daisy,” etc.). This mapping is important for interpretability during exploratory analysis, even though the clustering operates without relying on labels.

2.3 Exploratory Data Analysis (EDA)

The EDA portion focuses on understanding the dataset visually and structurally. This is done by first checking how many flower examples exist per category, and then visualizing a random example image from each class in a single row of subplots. This quick inspection, as seen in Figure 1, helps establish what kinds of cues might matter for clustering. In particular, we can see that flowers are distinguishable by color, texture, or petal shape cues, while backgrounds can vary significantly. Therefore, it becomes useful to consider features that emphasize structure rather than raw color alone.

Figure 1: Randomly Selected Flower Image of Each Class

2.4 Preprocessing

For preprocessing, the NumPy and PIL modules are used to convert each image into a consistent numeric format. NumPy is used for array conversion, normalization, reshaping, and concatenating features into one vector per image. PIL modules such as Image, ImageOps, and ImageFilter are used to standardize images to RGB, center-crop and resize them to a fixed size, and apply edge detection [1]. Matplotlib is then used to visualize intermediate outputs, such as RGB histograms and edge maps, to verify that the preprocessing steps are behaving as expected before clustering.

Figure 2: Dandelion Image

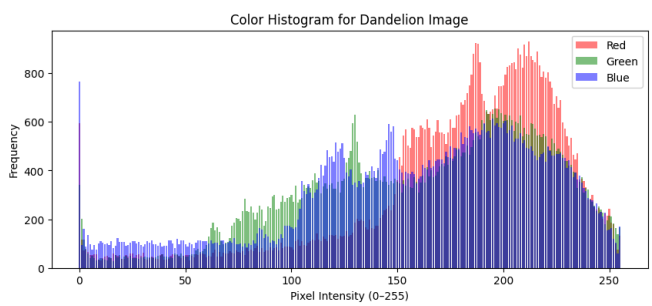

Figure 3: RGB Histogram of the Dandelion Image

Figure 4: Sunflower Image

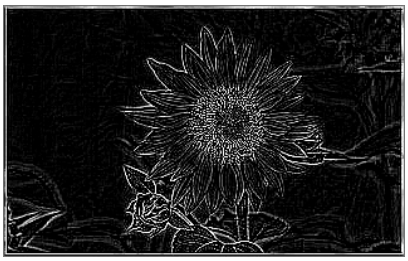

Figure 5: Edge Detection of the Sunflower Image

3 Model and Results

3.1 K-Means

The main model is K-Means clustering, which groups items by minimizing the distance between each data point and its assigned cluster centroid. In practice, K-Means assumes that similarity can be captured with a distance measure, such as Euclidean distance, in a feature space. If the extracted features capture meaningful visual similarity, the model has a chance to produce coherent clusters. However, if the features are noisy or dominated by irrelevant variation, clusters can be overlapping.

The process begins with feature scaling using Sklearn's StandardScaler. Since extracted feature vectors combine RGB values and edge information across thousands of dimensions, different dimensions can have different ranges and variances [2]. Standardization transforms each feature dimension to have zero mean and unit variance. Without scaling, dimensions with larger numeric ranges would dominate the distance computation and bias the clustering results.

After scaling, K-Means is applied. While in practice, to determine a “good” k-value for K-Means, experiments such as elbow curve are usually done, but in this exercise, five clusters are used as it aligns with the five known flower categories in the dataset. Importantly, even though this number mirrors the label count, the algorithm ignores the labels during fitting. The number of clusters defines how many groupings K-Means will attempt to discover [3].

As the feature space is very high dimensional, Principal Component Analysis (PCA) is used to make the clustering results easier to interpret. PCA is applied to the scaled feature matrix to project the data into a lower-dimensional space while preserving as much variance as possible. Although PCA is computed with five components, only the first two principal components are used for visualization. These two axes represent the directions of greatest variation in the data, providing a compact 2D summary of how images are distributed relative to one another [4]. The learned K-Means centroids are also projected into the same PCA space so they can be plotted alongside the data points. Figure 6 shows each image as a point in the two-dimensional PCA space, colored according to its K-Means cluster assignment. As a complement, Figure A1 shows some example flower images in each cluster as a visual representation of how our clustering performed.

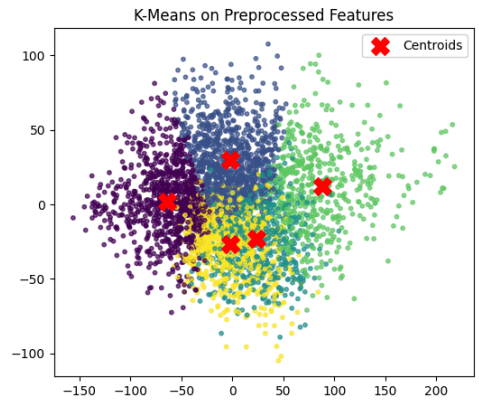

Figure 6: PCA 2D View of K-Means

3.2 Model Evaluation

Overall, The K-Means visualization does not show the full high-dimensional clustering directly, but it provides an intuitive summary of the structure discovered by K-Means. Overlapping regions suggest that some clusters are not perfectly separable under the current feature representation. This observation is consistent with the silhouette score of ~0.20, which indicates relatively weak cluster structure. In general, the silhouette coefficient ranges from -1 to 1, where values closer to 1 suggest that points are much closer to their own cluster than to other clusters, while values near 0 suggest that clusters overlap substantially. In the context of this exercise, this typically reflects the difficulty of clustering natural images with "vanilla clustering" where the performance of K-Means would rely heavily on our preprocessing.

4 Using Pre-Trained Convolutional Neural Network (CNN) for Feature Extraction

4.1 VGG16

As we saw before, "vanilla" clustering on flower images proved to be quite weak. Therefore, the motivation for using a pre-trained CNN, particularly VGG16, in an unsupervised clustering setting is that its convolutional layers learn hierarchical visual patterns. Early layers respond to simple structures like edges and color contrasts, while deeper layers respond to more complex shapes and textures [5]. As a result, VGG16 can provide a more meaningful description of an image than raw pixels or simple edge filters which was done earlier.

By default, VGG16 loads the full network, including a classification head that was trained for ImageNet [5]. However, instead of using the model for classification, the classification head is removed. Essentially, the model is repurposed as a feature extractor by redefining the model output. Specifically, a new model is created where the inputs are the same

as VGG16's original inputs, but the outputs come from the second-to-last layer, fc2, which produces a 4096-dimensional vector for each image. This choice is going to be used for downstream tasks like clustering.

4.2 Preprocessing

In the VGG16 framework, we use the same preprocessing function as we did with the vanilla clustering. Each image is resized to the model's expected input size and then passed through the model's built-in preprocessing function, which normalizes pixel values using the same convention VGG16 was trained with. After that, the image is wrapped into a single-item batch and a forward pass is run to produce a fixed-length feature vector. Collecting these vectors for all images produces a feature matrix that can be clustered with the same K-Means framework as before, except now using CNN embeddings instead of handcrafted RGB and edge features.

4.3 VGG16 Model Evaluation

Compared to the vanilla clustering based on handcrafted RGB and edge features, the clustering based on VGG16 features produces groups that appear more compact and better localized in the 2D PCA projection, as seen in Figure 7. While some overlap still exists, the boundaries between clusters are visually sharper, suggesting that the feature representation captures more discriminative information. Furthermore, with the VGG16-based feature representation, the silhouette score increases to ~0.27, which suggests a modest improvement in cluster separation and compactness. Figure A2 provides example images in each cluster, where we can see how this improvement is consistent with the idea that deep CNN embeddings encode higher-level visual patterns.

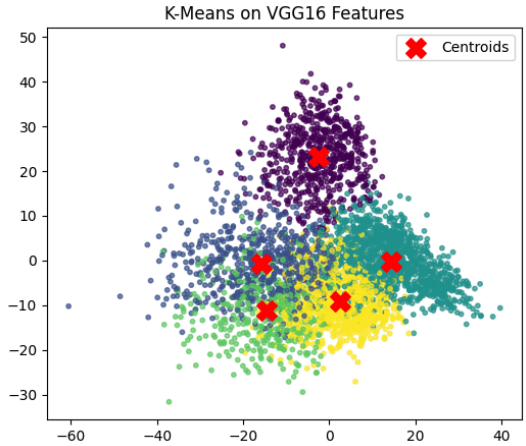

Figure 7: PCA 2D View of K-Means (VGG16 Features)

References:

[1] *Pillow: Python Imaging Library (Fork)*. Python. *PyPI*, https://python-pillow.github.io. Accessed 19 Dec. 2025.

[2] "StandardScaler." *Scikit-Learn*, https://scikit-learn/stable/modules/generated/sklearn.preprocessing.StandardScaler.html. Accessed 19 Dec. 2025.

[3] "KMeans." *Scikit-Learn*, https://scikit-learn/stable/modules/generated/sklearn.cluster.KMeans.html. Accessed 19 Dec. 2025.

[4] "PCA." *Scikit-Learn*, https://scikit-learn/stable/modules/generated/sklearn.decomposition.PCA.html. Accessed 19 Dec. 2025.

[5] *Vgg16 — Torchvision Main Documentation*. https://docs.pytorch.org/vision/main/models/generated/torchvision.models.vgg16.html. Accessed 19 Dec. 2025.

Appendix:

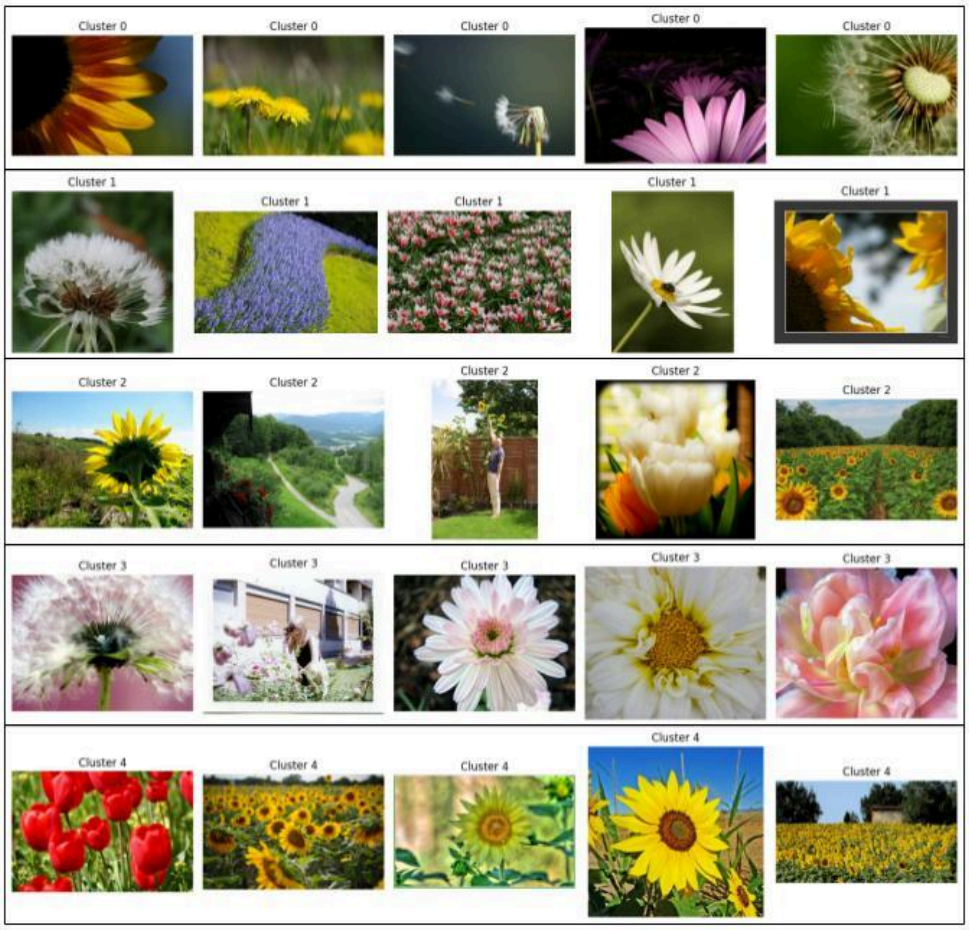

Figure A1: Example Flower Images for Each Cluster

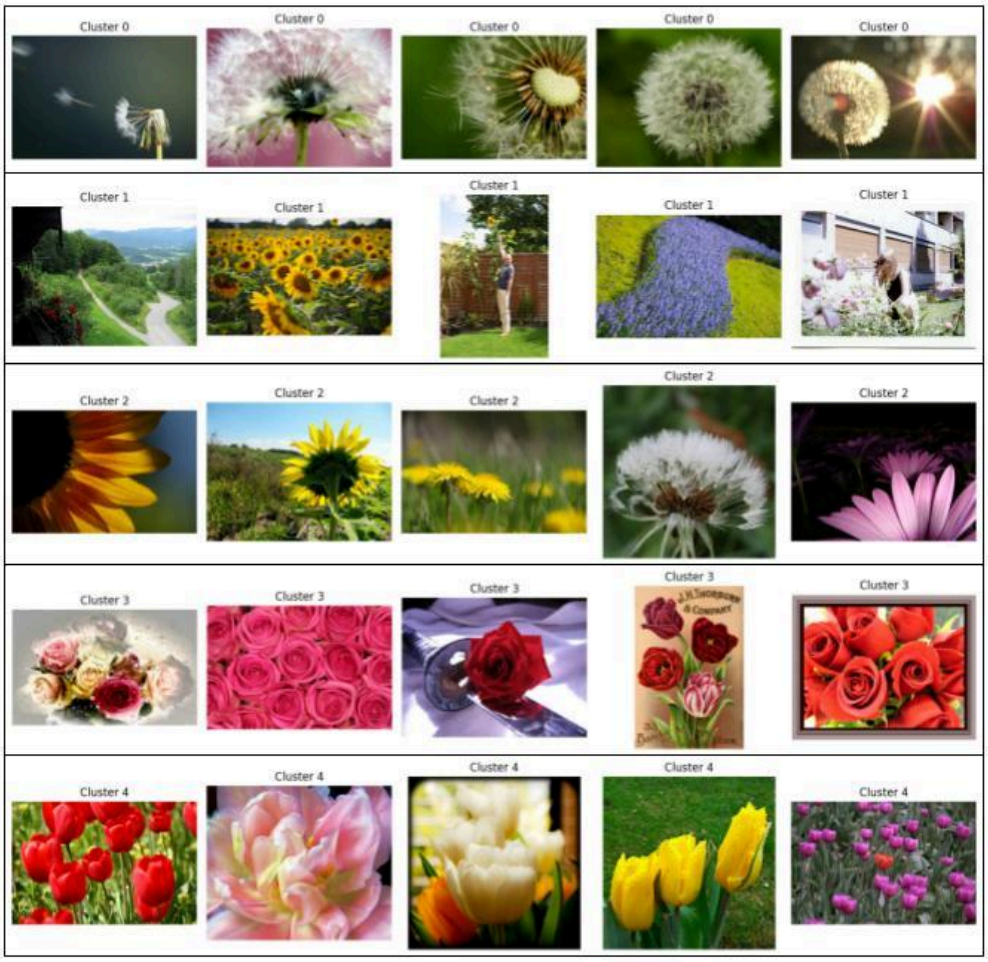

Figure A2: Example Flower Images for Each Cluster (VGG16 Features)